Studio Nx

JS 291 — Subtractive Synth

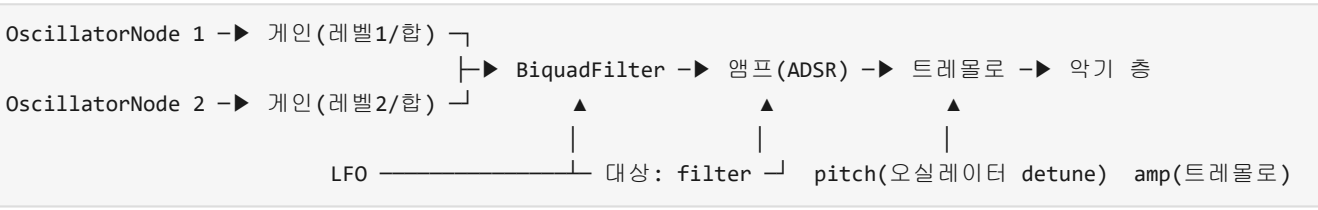
2026-10-19

JavaScript 291 — Subtractive Synth (`audio/synth.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/synth.js`
작업	S1-11a 재생 오디오 그래프: 감산 신스가 노트 하나를 소리로 냄 (FR-242)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 290 재생 세션, 다음 292 노트 재생과 악기 층

1. 하는 일

노트 하나를 "오실레이터 2개, 필터, 앰프 엔벨로프(ADSR), LFO" 그래프로 만들어 울립니다. 파라미터는 `plugins.param_schema`의 key를 그대로 읽습니다. 계산 부분은 순수 함수로 분리했습니다.



2. 언제 쓰나

쓰는 곳	하는 일
`InstrumentRack.playNote` (292)	트랙에 붙은 `synth_sub` 층마다 노트가 올 때 `playSynthNote`를 부릅니다.
WAV 출력 (S1-11b 예정)	같은 함수를 `OfflineAudioContext`에서 불러 노트를 렌더링합니다.

3. 코드

3.1 순수 함수

```
export function noteToFreq(pitch) {
  return 440 * Math.pow(2, (Number(pitch) - 69) / 12);
}
```

```
export function resolveParams(schema, params) {
  const out = {};
  (schema || []).forEach(function (item) {
    let value = item.default;
    if (params && params[item.key] !== undefined && params[item.key] !== null) value = params[item.key];
    if (typeof item.min === 'number' && typeof item.max === 'number' && typeof value === 'number') {
      value = Math.max(item.min, Math.min(item.max, value));
    }
    out[item.key] = value;
  });
  return out;
}
```

```
export function oscType(wave) {
  if (wave === 'saw') return 'sawtooth';
  if (wave === 'sine' || wave === 'triangle' || wave === 'square') return wave;
  return 'sawtooth';
}
```

```
export function envelopeLevel(t, a, d, s, peak) {
  if (t <= 0) return 0;
  if (a > 0 && t < a) return peak * (t / a);
  const td = t - a;
  if (d > 0 && td < d) return peak * (1 - (1 - s) * (td / d));
  return peak * s;
}
```

3.2 playSynthNote

```
export function playSynthNote(ctx, dest, p, pitch, velocity, when, duration) {
  const freq = noteToFreq(pitch);
  const peak = 0.3 * Math.max(0, Math.min(127, velocity)) / 127;
  const a = Number(p.env_attack) || 0;
  const d = Number(p.env_decay) || 0;
  const s = p.env_sustain != null ? Number(p.env_sustain) : 1;
  const r = Math.max(0.005, Number(p.env_release) || 0);
  const dur = Math.max(0.01, duration);

  // 앰프 엔벨로프
  const amp = ctx.createGain();
  amp.gain.setValueAtTime(0, when);
  if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
  else amp.gain.setValueAtTime(peak, when);
  if (d > 0 && dur > a) amp.gain.linearRampToValueAtTime(peak * s, when + Math.min(dur, a + d));
  const atRelease = envelopeLevel(dur, a, d, s, peak);
  amp.gain.setValueAtTime(atRelease, when + dur);
  amp.gain.linearRampToValueAtTime(0, when + dur + r);
  const end = when + dur + r + 0.02;

  // 필터
  const filter = ctx.createBiquadFilter();
  filter.type = p.filter_type === 'highpass' || p.filter_type === 'bandpass' ? p.filter_type : 'lowpas
s';
  filter.frequency.value = Math.max(20, Math.min(20000, Number(p.cutoff) || 1200));
  filter.Q.value = 0.5 + Math.max(0, Math.min(1, Number(p.resonance) || 0)) * 15;

  // 트레몰로(LFO → 앰프)는 앰프 뒤에 따로 두어 엔벨로프와 곱해지게 함
  const trem = ctx.createGain();
  filter.connect(amp);
  amp.connect(trem);
  trem.connect(dest);

  // 오실레이터 2개 (레벨은 합이 1이 되게 나눠 소리가 겹쳐도 커지지 않게 함)
  const l1 = Math.max(0, Number(p.osc1_level) || 0);
  const l2 = Math.max(0, Number(p.osc2_level) || 0);
  const sum = l1 + l2 > 0 ? l1 + l2 : 1;
  const oscs = [];
  [[l1, p.osc1_wave, Number(p.osc1_detune) || 0, 0], [l2, p.osc2_wave, Number(p.osc2_detune) || 0, Numbe
```

```
r(p.osc2_semi) || 0]]
    .forEach(function (cfg) {
      if (!(cfg[0] > 0)) return;
      const osc = ctx.createOscillator();
      osc.type = oscType(cfg[1]);
      osc.frequency.value = freq * Math.pow(2, cfg[3] / 12);
      osc.detune.value = cfg[2];
      const g = ctx.createGain();
      g.gain.value = cfg[0] / sum;
      osc.connect(g);
      g.connect(filter);
      osc.start(when);
      osc.stop(end);
      oscs.push(osc);
    });

  // LFO (깊이 0 이거나 대상이 off 면 만들지 않음)
  const depth = Math.max(0, Math.min(1, Number(p.lfo_depth) || 0));
  const target = p.lfo_target;
  const extra = [];
  if (depth > 0 && (target === 'pitch' || target === 'filter' || target === 'amp')) {
    const lfo = ctx.createOscillator();
    lfo.frequency.value = Math.max(0.01, Number(p.lfo_rate) || 1);
    const lg = ctx.createGain();
    lfo.connect(lg);
    if (target === 'pitch') {
      lg.gain.value = depth * 100;                       // 최대 ±100 cent
      oscs.forEach(function (o) { lg.connect(o.detune); });
    } else if (target === 'filter') {
      lg.gain.value = depth * filter.frequency.value;   // 컷오프의 ±(깊이 × 컷오프)
      lg.connect(filter.frequency);
    } else {
      trem.gain.value = 1 - depth / 2;                  // 1 - 깊이/2 를 중심으로 ±깊이/2 만큼 흔듦
      lg.gain.value = depth / 2;
      lg.connect(trem.gain);
    }
    lfo.start(when);
    lfo.stop(end);
    extra.push(lfo);
  }

  return {
    // 정지 · 루프 되감기 때 소리를 끊음 (뚝 끊기는 소리를 줄이려고 아주 짧은 릴리스를 줌)
    stop: function (at) {
      const t = Math.max(at, ctx.currentTime);
      try { amp.gain.cancelScheduledValues(t); amp.gain.setTargetAtTime(0, t, 0.01); } catch (e) { /* 무
시 */ }
      oscs.concat(extra).forEach(function (o) { try { o.stop(t + 0.08); } catch (e) { /* 이미 끝남 */ }
});
    },
  };
}
```

4. 최신 문법으로 쓰면

```
// resolveParams 를 구조 분해와 nullish 병합으로 쓴 모양
export const resolveParams = (schema = [], params = null) =>
  Object.fromEntries(schema.map(({ key, min, max, default: def }) => {
    let value = params?.[key] ?? def;
    if (typeof min === 'number' && typeof max === 'number' && typeof value === 'number') {
      value = Math.max(min, Math.min(max, value));
    }
    return [key, value];
  }));
```

전통 문법 (실제 코드)	최신 문법
`params && params[item.key] !== undefined && params[item.key] !== null`	`params?.[item.key] ?? item.default`
`forEach` 로 객체에 담기	`Object.fromEntries(map(...))`
`item.default`	구조 분해 `{ default: def }`

5. 왜 이렇게

선택	이유
저장된 값을 `param_schema`의 범위로 자릅니다	잘못 저장된 값(예: 음수 어택)이 소리를 망가뜨리지 않게 합니다.
두 오실레이터의 레벨을 합으로 나눕니다	레벨을 둘 다 1로 두어도 소리가 두 배로 커지지 않습니다. 합이 1이 되게 정규화합니다.
릴리스 직전 값을 계산합니다 (`envelopeLevel`)	노트가 어택이나 디케이 중에 끝나도, 그 순간의 값에서 0으로 이어져야 "탁" 하는 잡음이 없습니다.
LFO 대상을 세 가지로 나눴습니다	피치는 `detune`(최대 ±100센트), 필터는 컷오프(±깊이 곱하기 컷오프), 앰프는 트레몰로(1 - 깊이/2를 중심으로 ±깊이/2)를 흔듭니다.
트레몰로를 앰프 뒤 별도 게인으로 뒀습니다	엔벨로프와 곱해져야 해서 같은 게인에 이으면 서로 덮어씁니다.
오실레이터는 노트마다 새로 만듭니다	한 번 `stop`한 오실레이터는 다시 쓸 수 없습니다. 가볍게 만들 수 있게 설계된 노드입니다.
`stop`은 짧은 릴리스로 끊습니다	재생을 멈출 때 소리가 갑자기 끊기면 클릭이 납니다. 10ms 시간 상수로 줄이고 80ms 뒤 멈춥니다.
동시 발음 수에 제한이 없습니다	지금은 단순함을 우선했습니다. 노트가 매우 많아지면 제한을 둘 수 있습니다.

6. 시험

- `docs/handoff/s1-11a_tests/synth_notes_unit.mjs`는 다음을 확인합니다. A4는 440Hz이고 A5는 880Hz입니다. 범위를 벗어난 컷오프는 20000으로 잘립니다. 스키마에 없는 키는 버려지고, `null` 값은 기본값을 씁니다. 엔벨로프는 어택 중 절반, 디케이 중 75%, 서스테인에서 50%입니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 실제 화면에서 A4 노트가 소리로 나오는지 확인합니다. 피크가 0.05보다 크고 0.5보다 작고, 뮤트하면 무음이 됩니다.
- 시험 중 발견한 일: 처음에는 소리가 거의 0이었는데, 그 트랙의 기존 Synth에 컷오프 20Hz가 저장되어 있었고 트랙 볼륨도 -21.5dB였기 때문입니다. 코드는 정상이었고, 시험이 기본값의 새 Synth를 붙이도록 고

쳤습니다.